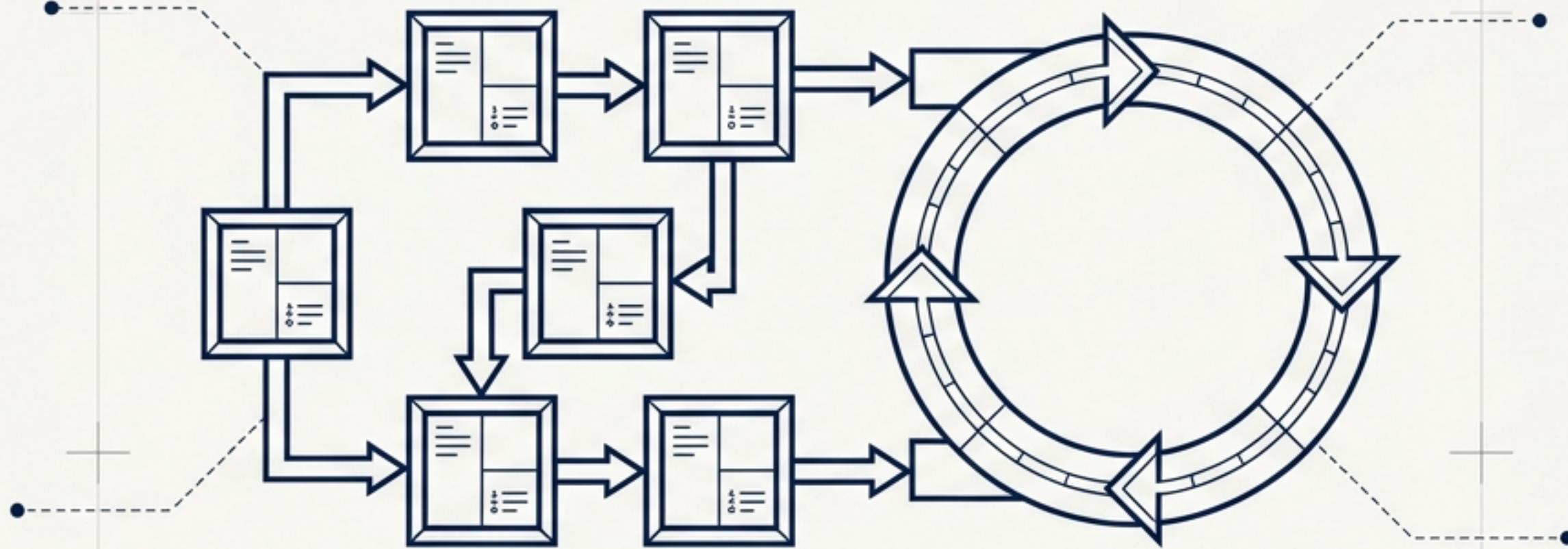


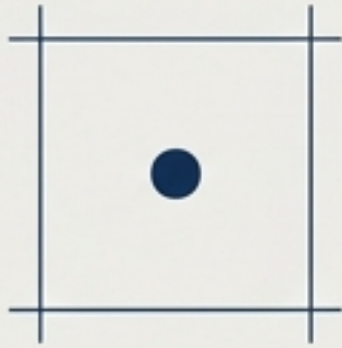
[ENTORNO_PL/SQL]



Tipos de Datos Compuestos y Cursores

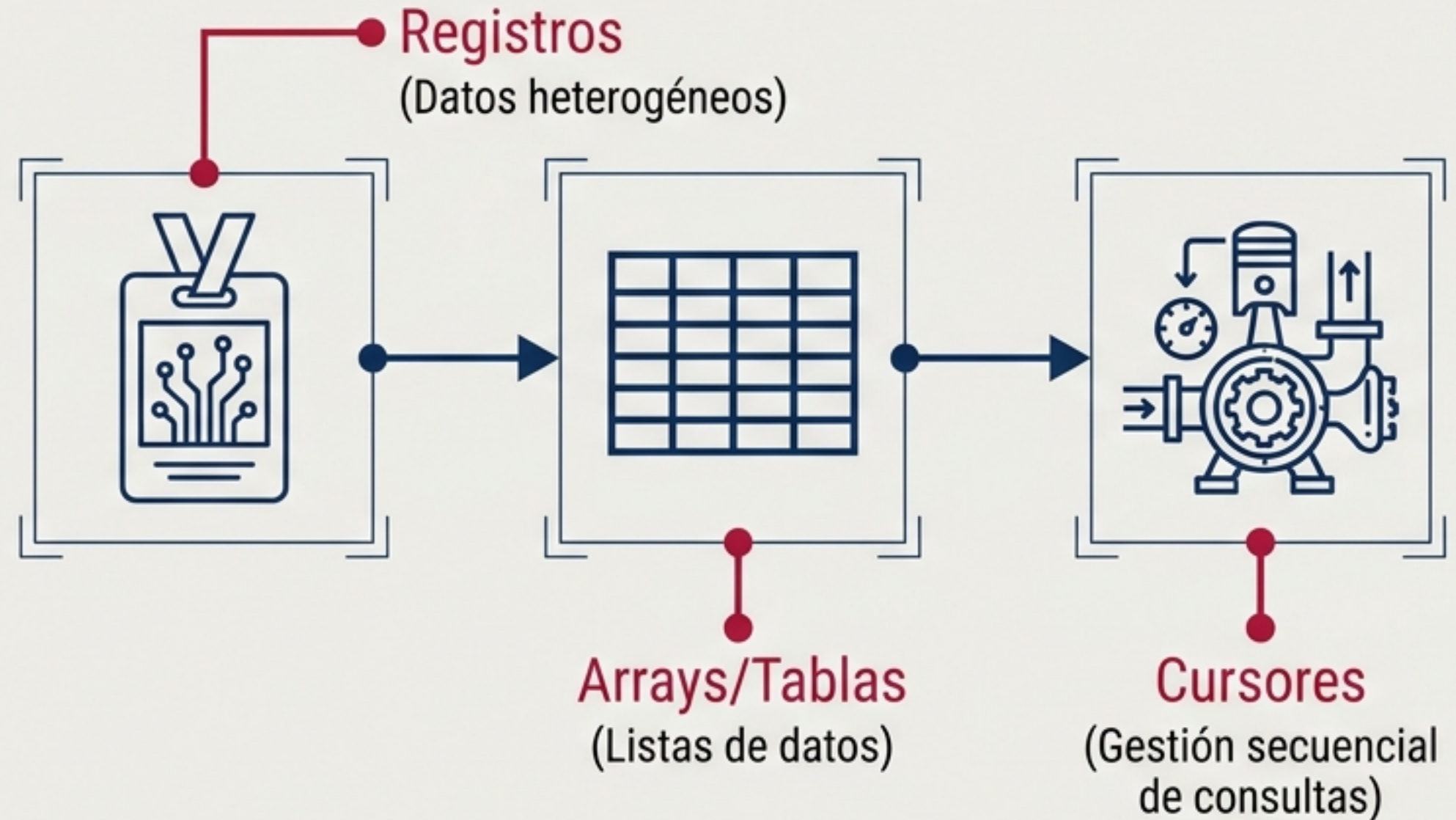
Dominando las estructuras fundamentales en memoria y los motores de extracción de resultados en bases de datos.

La Limitación Simple



Las variables tradicionales son insuficientes para modelar entidades complejas o manejar múltiples resultados de consultas SQL.

El Kit de Herramientas Avanzado



Registros: La Fila en Memoria

Un conjunto de datos relacionados y organizados en campos, **idéntico a una fila** de una tabla de base de datos.



SINTAXIS

```
TYPE registro IS RECORD (  
  Campo1 TipoDatos1 [NOT NULL] := Expresion1,  
  Campo2 TipoDatos2 [NOT NULL] := Expresion2,  
  ...  
  CampoN TipoDatosN [NOT NULL] := ExpresionN  
);
```

%TYPE (hereda tipo de un campo)

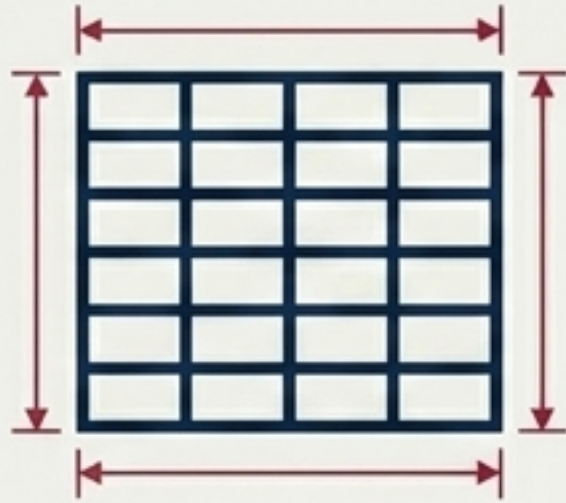
%ROWTYPE (hereda estructura de una fila completa)

EJEMPLO

```
DECLARE  
  TYPE Persona IS RECORD (  
    CODIGO INTEGER(2),  
    NOMBRE VARCHAR(20),  
    EDAD NUMBER(3)  
  );  
  Juan Persona;  
BEGIN  
  Juan.CODIGO := 1;  
  Juan.NOMBRE := 'Juan';  
  Juan.EDAD := 33;  
  -- INSERT INTO PRUEBA VALUES Juan;  
END;
```

Regla: No se pueden realizar operaciones INSERT directamente sobre ellos sin desglosar, pero sí consultas SELECT.

MATRIZ DE DECISIÓN: El Ecosistema de Colecciones



VARRAY

- Límite fijo predefinido.
- Acceso secuencial por índice numérico.
- Almacenamiento contiguo.



Array Asociativo

- Tamaño dinámico.
- Pares Clave-Valor.
- Índices numéricos o cadenas (VARCHAR2).
- Búsquedas ultrarrápidas en memoria.

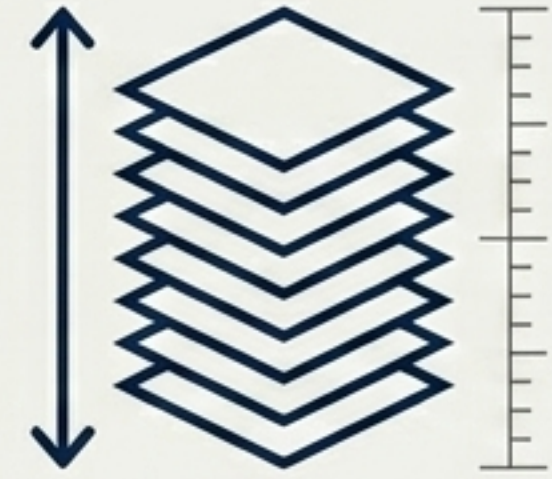
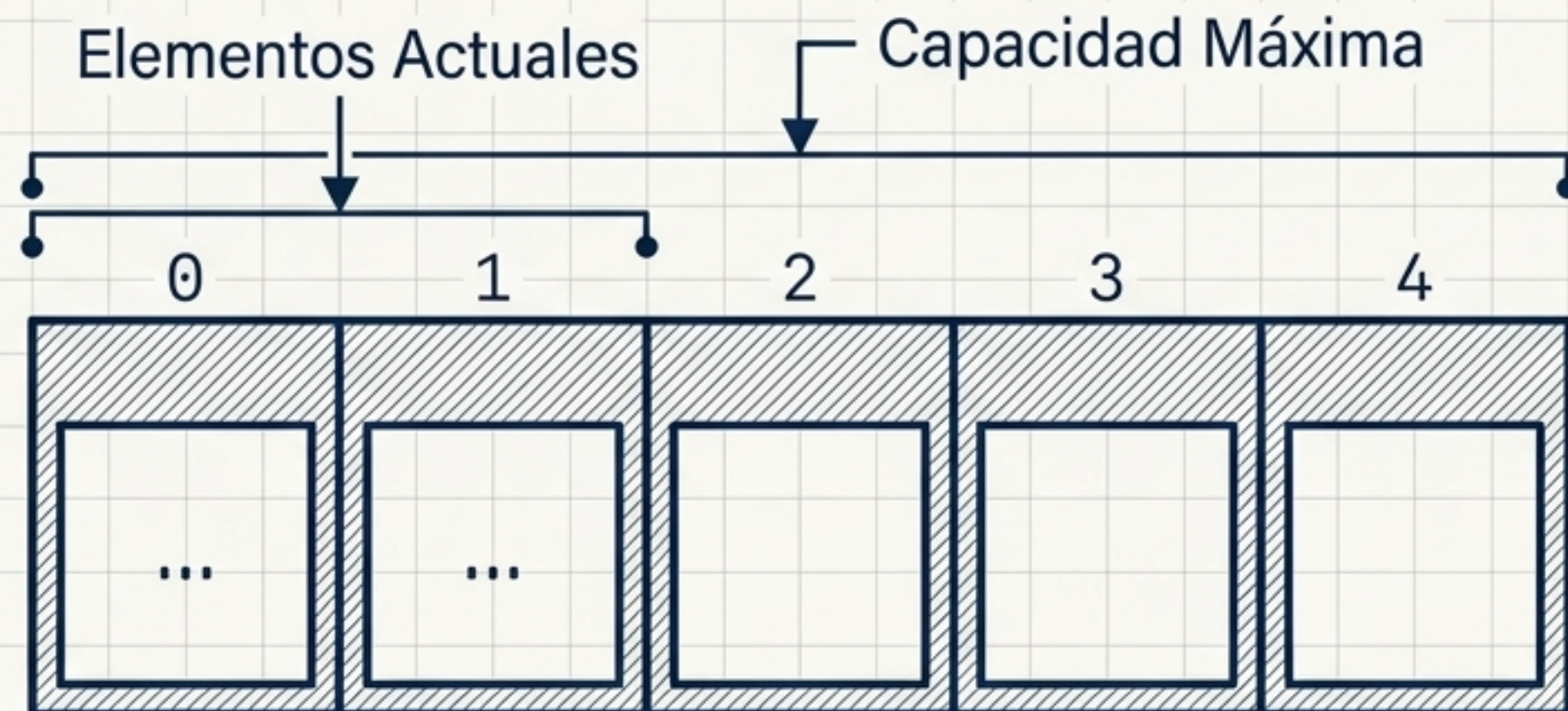


Tabla Anidada

- Tamaño totalmente dinámico.
- Permite borrado de elementos intermedios sin reordenar.
- Ideal para bases de datos extensas.

VARRAYs: Arquitectura de Capacidad Fija

Colecciones ordenadas de elementos del mismo tipo. Empiezan vacías y requieren inicialización.



```
TYPE nombreArray IS  
  VARRAY (capacidad_máxima) OF  
  tipo_de_elementos [NOT NULL];
```



Importante: No soporta tipos de datos como BOOLEAN, LONG, o NATURAL.

Colecciones Dinámicas: Elasticidad Total

Arrays Asociativos

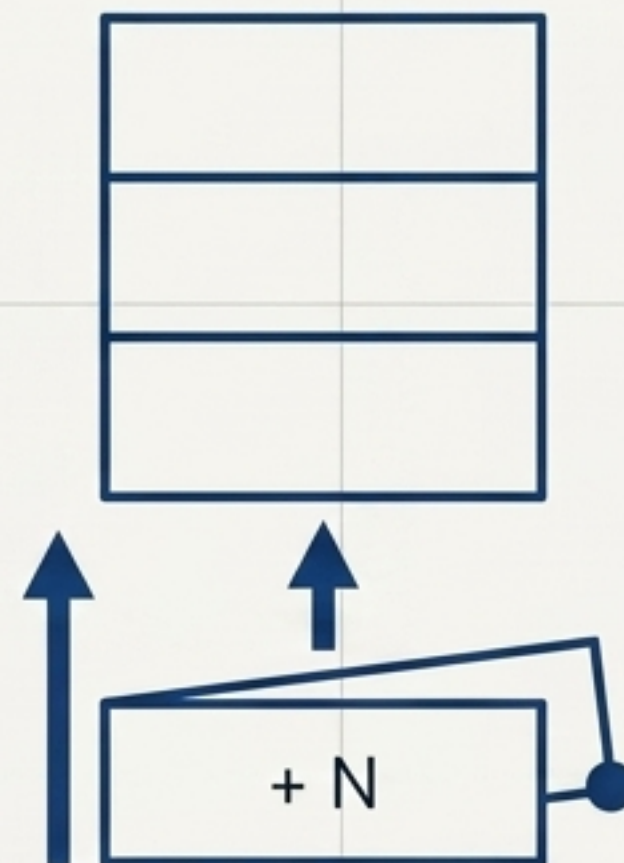
Perfectos para cachés en memoria.
Iniciación directa clave-valor.



```
DECLARE  
  TYPE t_poblacion_ciudad  
    IS TABLE OF NUMBER INDEX BY VARCHAR2(100);  
  v_poblacion_ciudad t_poblacion_ciudad;
```

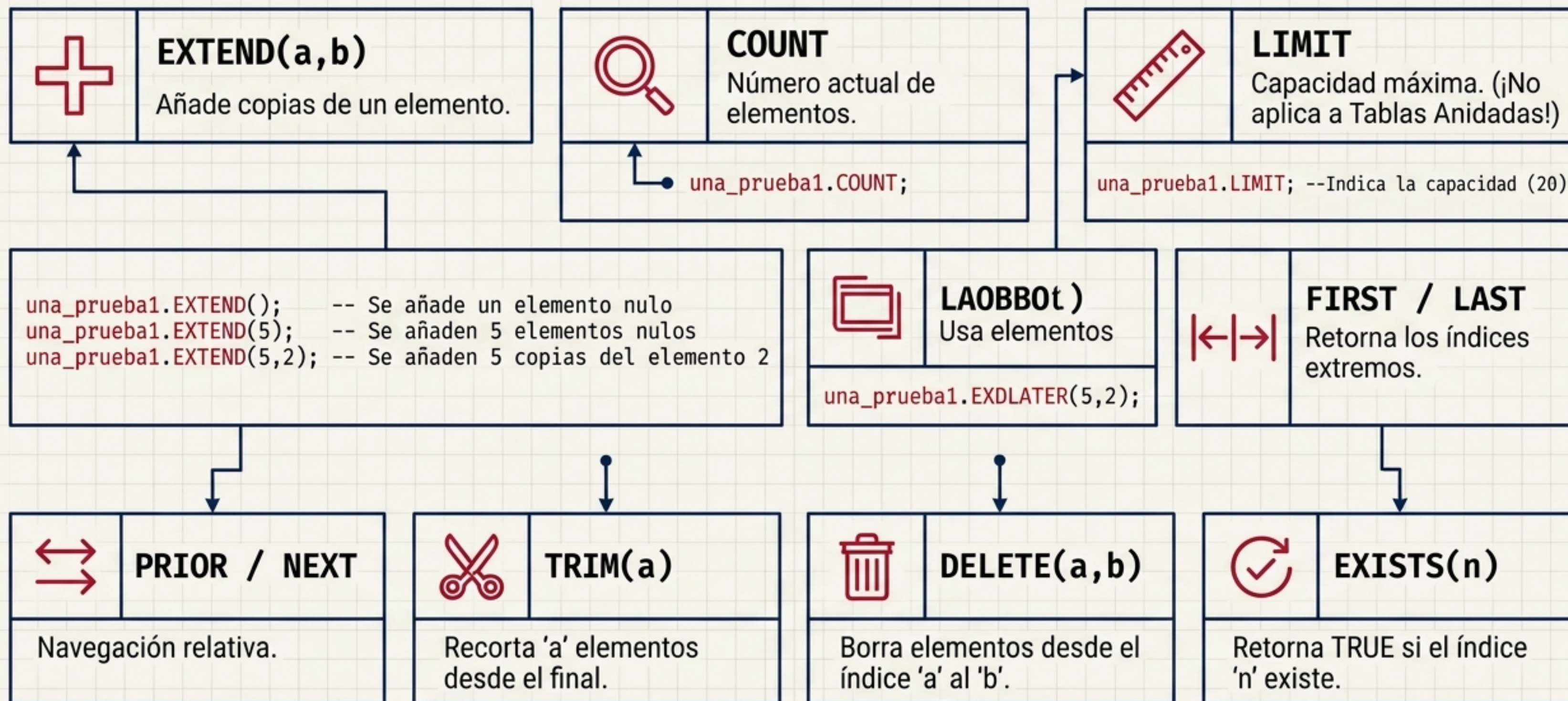
Tablas Anidadas

Escalabilidad absoluta sin límite estricto
predefinido.



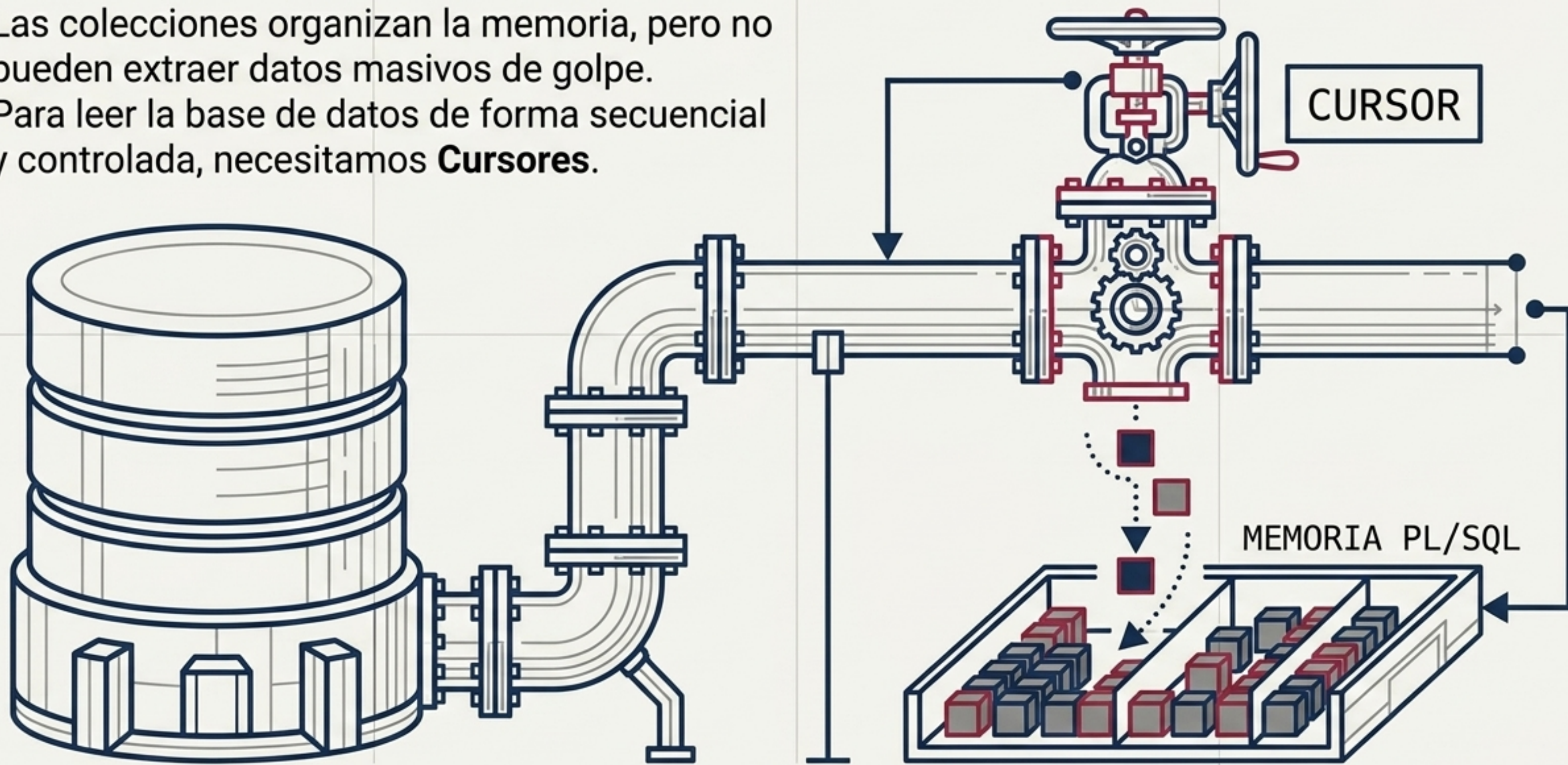
```
TYPE nombreTabla IS TABLE OF tipo_de_elementos [NOT NULL];
```


La Consola de Métodos de Colección

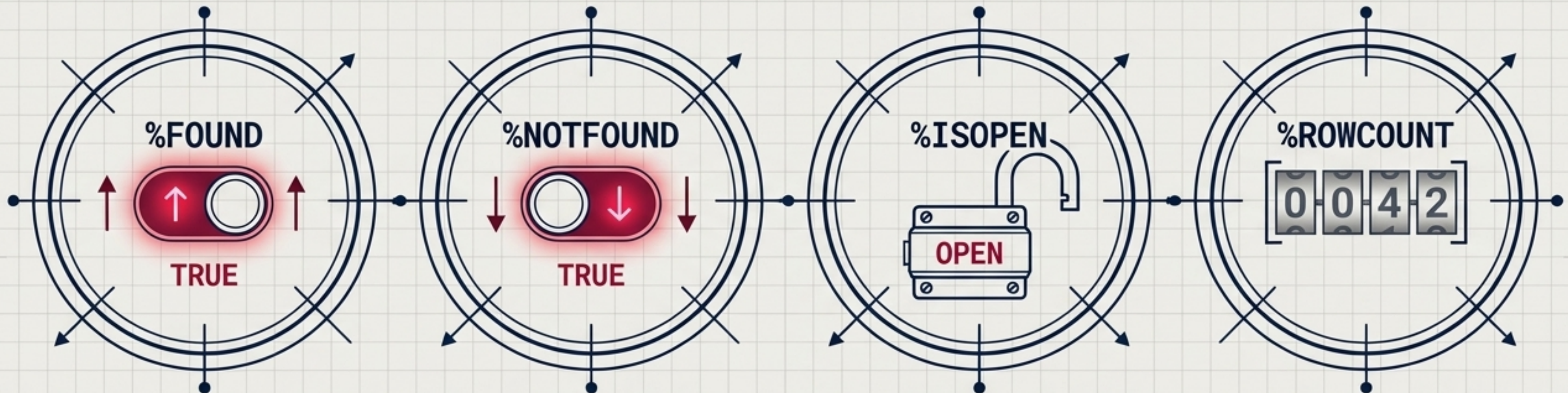


¿Y si la consulta devuelve miles de filas?

Las colecciones organizan la memoria, pero no pueden extraer datos masivos de golpe.
Para leer la base de datos de forma secuencial y controlada, necesitamos **Cursores**.



Panel de Diagnóstico de Cursores (Propiedades)



%FOUND

TRUE si el último FETCH extrajo una fila.

%NOTFOUND

TRUE si ya no quedan más filas por extraer. Ideal para salir de bucles.

```
EXIT WHEN cursor_emp%NOTFOUND;
```

%ISOPEN

TRUE si el cursor está listo para leer. Evita errores de acceso.

```
IF NOT cursor_emp%ISOPEN THEN  
  OPEN cursor_emp;  
END IF;
```

%ROWCOUNT

Contador histórico de filas procesadas.

```
WHILE cursor_emp%FOUND LOOP  
  DBMS_OUTPUT.PUT_LINE('Procesadas:  
    || cursor_emp%ROWCOUNT);  
END LOOP;
```


Cursores: Automatización vs. Control Total



Implícitos

- Creados automáticamente por el SGBD para DML (SELECT INTO, INSERT, UPDATE, DELETE).
- Siempre operan bajo el nombre interno SQL.

```
BEGIN
  UPDATE emp SET sal = sal * 1.1 WHERE deptno = 10;
  IF SQL%FOUND THEN
    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' filas actualizadas.');
```

```
ELSE
  DBMS_OUTPUT.PUT_LINE('No se han actualizado filas.');
```

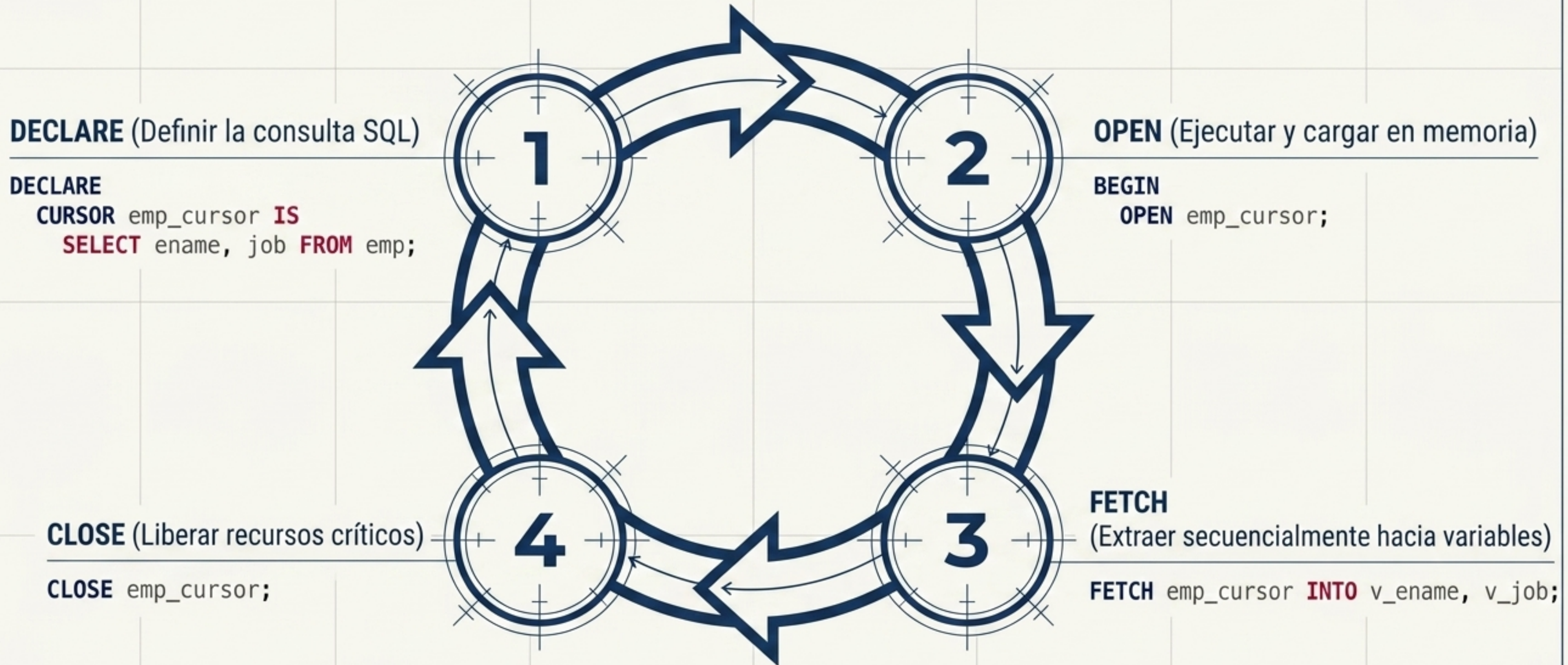
```
END IF;
END;
```



Explícitos

- Definidos y gestionados íntegramente por el usuario.
- Obligatorios para consultas SELECT que devuelven múltiples filas. Residen en la sección DECLARE.

El Ciclo de Vida del Cursor Explícito



Superpoderes del Cursor: Parámetros y Bloqueos



Cursores Parametrizados

Inyección de variables en la apertura para reutilizar consultas. Filtra resultados dinámicamente basados en un valor específico.

```
DECLARE
  CURSOR emp_cursor (p_deptno NUMBER) IS
    SELECT ename, job FROM emp
    WHERE deptno = p_deptno;
```



Actualización Segura

Uso de FOR UPDATE NOWAIT para bloquear filas en entornos multiusuario, garantizando consistencia. Actualización precisa mediante WHERE CURRENT OF.

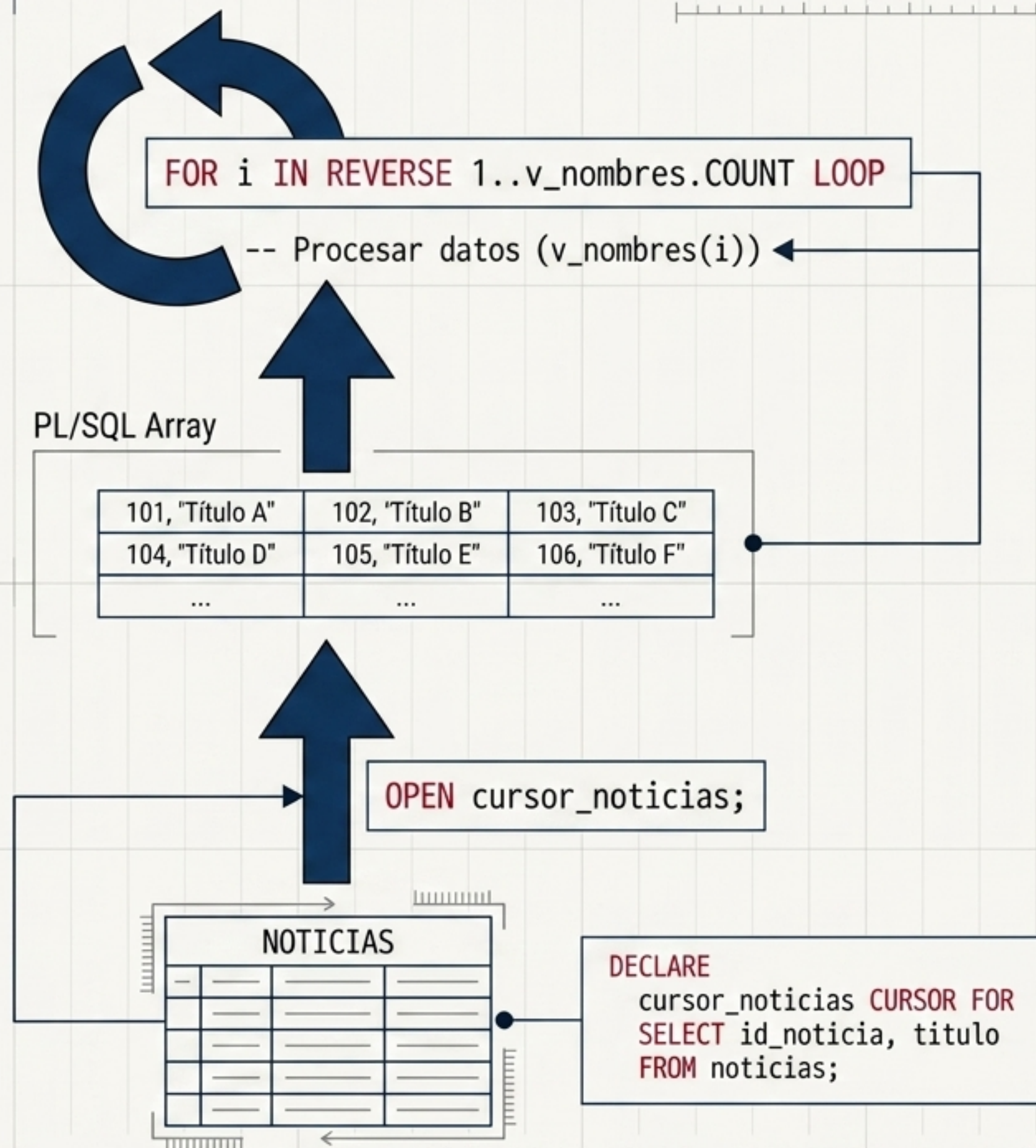
```
DECLARE
  CURSOR cursor_empleado IS
    SELECT id_empleado, nombre, apellidos, salario
    FROM EMPLEADOS
    WHERE departamentos.id_empleado = empleados.id_empleado
    FOR UPDATE OF salario NOWAIT;
BEGIN
  LOOP
    FETCH cursor_empleado INTO v_id_empleado, v_salario;
    EXIT WHEN cursor_empleado%NOTFOUND;

    UPDATE EMPLEADOS
    SET salario = v_salario * 1.1
    WHERE CURRENT OF cursor_empleado;
  END LOOP;

  CLOSE cursor_empleado;
  COMMIT;
END;
```


Arquitectura en Acción: Extracción e Iteración

Integración total: Un cursor extrae la información masiva desde la tabla, la deposita ordenadamente en variables o colecciones en memoria, y estructuras iterativas procesan el resultado final.



Master Cheat Sheet: Estructuras PL/SQL

